🧠 Recurrent Neural Networks (RNN) ve LSTM

Artık PyTorch ve derin öğrenme (Deep Learning) konusunda biraz deneyim kazandığına göre, sana **Recurrent Neural Networks (RNN)** ve **Long Short-Term Memory (LSTM)** konularını anlatacağım.

1. Recurrent Neural Networks (RNN)

- **Amaç:** Sıralı verilerden (sequences of data) öğrenmek için özel olarak tasarlanmıştır.
- **Çalışma Mantığı:** Gizli durumu (*hidden state*) dizinin bir adımından bir sonrakine aktarır ve bunu o anki girdi (*input*) ile birleştirerek işler.

2. Long Short-Term Memory (LSTM)

- **İlişki:** RNN'lerin geliştirilmiş bir versiyonudur.
- **Kullanım Alanı:** Sinir ağımızın;
  - Yakın geçmişteki olayları hatırlaması,
  - Ve çok uzun zaman önceki olayları hatırlaması arasında geçiş yapması gerektiğinde oldukça kullanışlıdır.

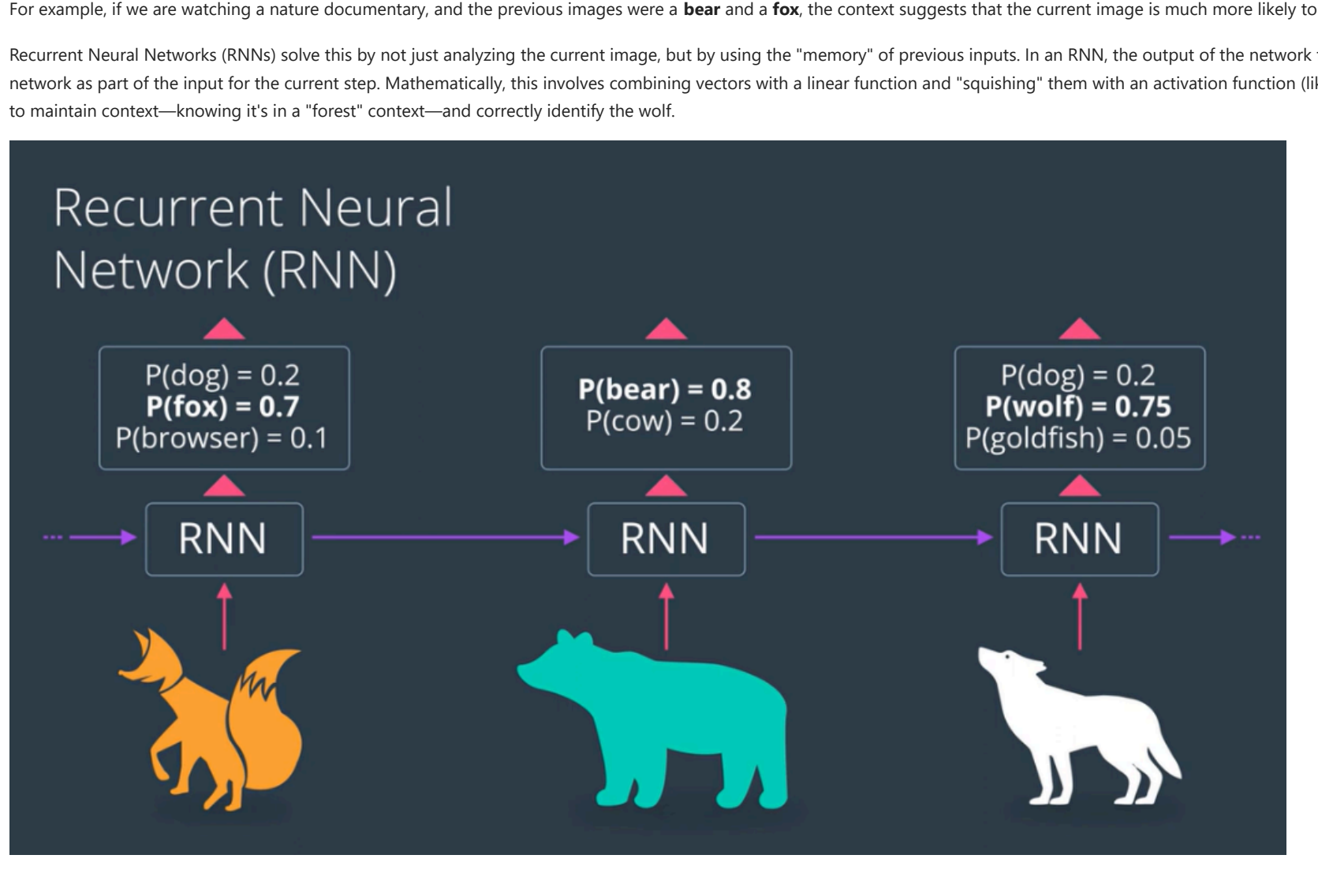
RNN vs. LSTM

The Logic of Recurrent Neural Networks (RNNs)

Imagine we have a standard neural network trained to recognize images. If we feed it an image, it might guess that it's most likely a "dog", with a small probability of being a "wolf". But what if the image typically appears in a sequence?

For example, if we are watching a nature documentary, and the previous images were a **bear** and a **fox**, the context suggests that the current image is much more likely to be a **wolf** than a domestic dog.

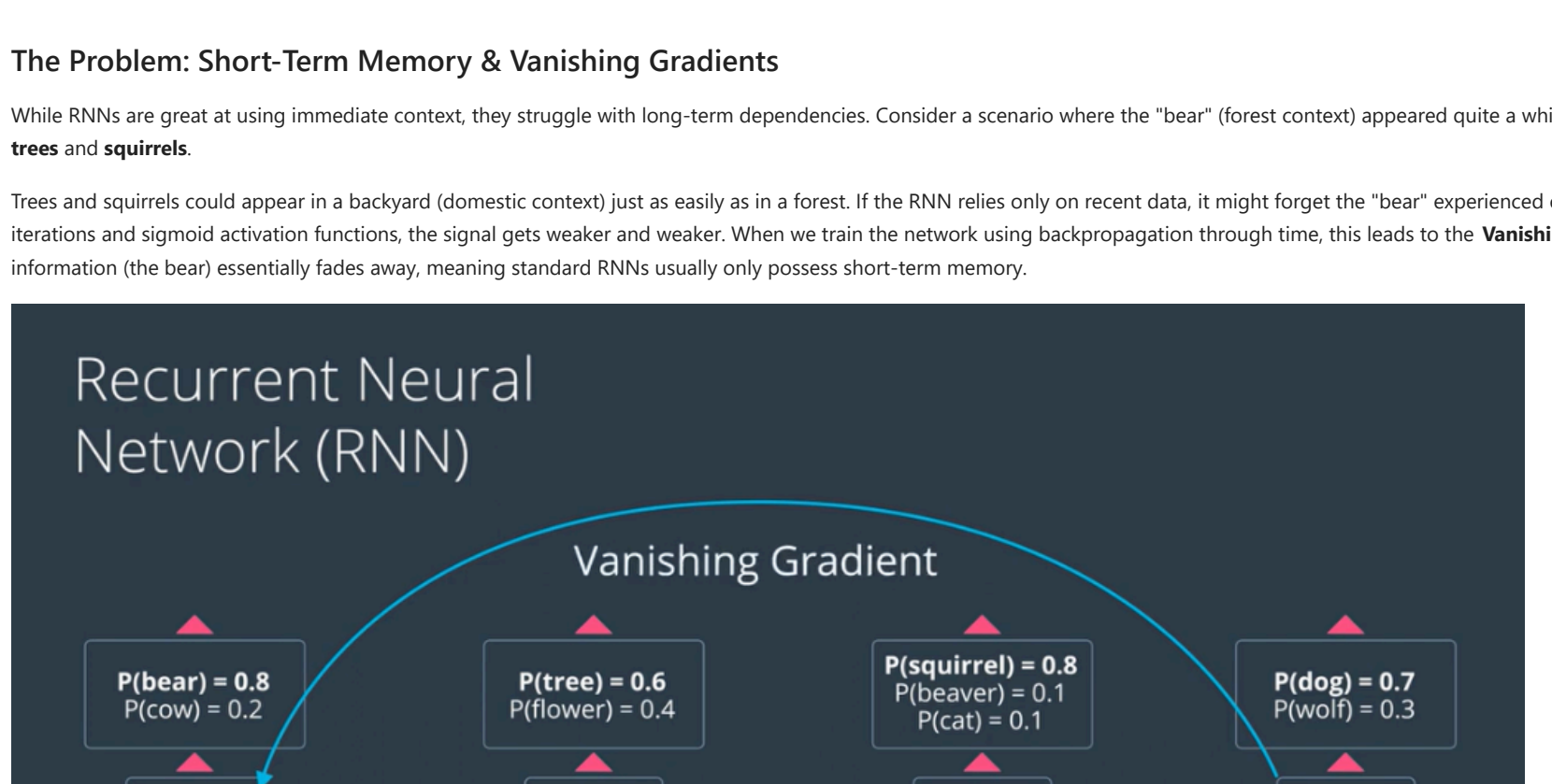
Recurrent Neural Networks (RNNs) solve this by not just analyzing the current image, but by using the "memory" of previous inputs. In an RNN, the output of the network from the previous step is fed back into the network as part of the input for the current step. Mathematically, this involves combining vectors with a linear function and "squishing" them with an activation function (like sigmoid or tanh). This allows the network to maintain context—knowing it's in a "forest" context—and correctly identify the wolf.



The Problem: Short-Term Memory & Vanishing Gradients

While RNNs are great at using immediate context, they struggle with long-term dependencies. Consider a scenario where the "bear" (forest context) appeared quite a while ago, and the most recent images were just **trees** and **squirrels**.

Trees and squirrels could appear in a backyard (domestic context) just as easily as in a forest. If the RNN relies only on recent data, it might forget the "bear" experienced earlier. As information passes through many iterations and sigmoid activation functions, the signal gets weaker and weaker. When we train the network using backpropagation through time, this leads to the **Vanishing Gradient Problem**. The "long-term" information (the bear) essentially fades away, meaning standard RNNs usually only possess short-term memory.



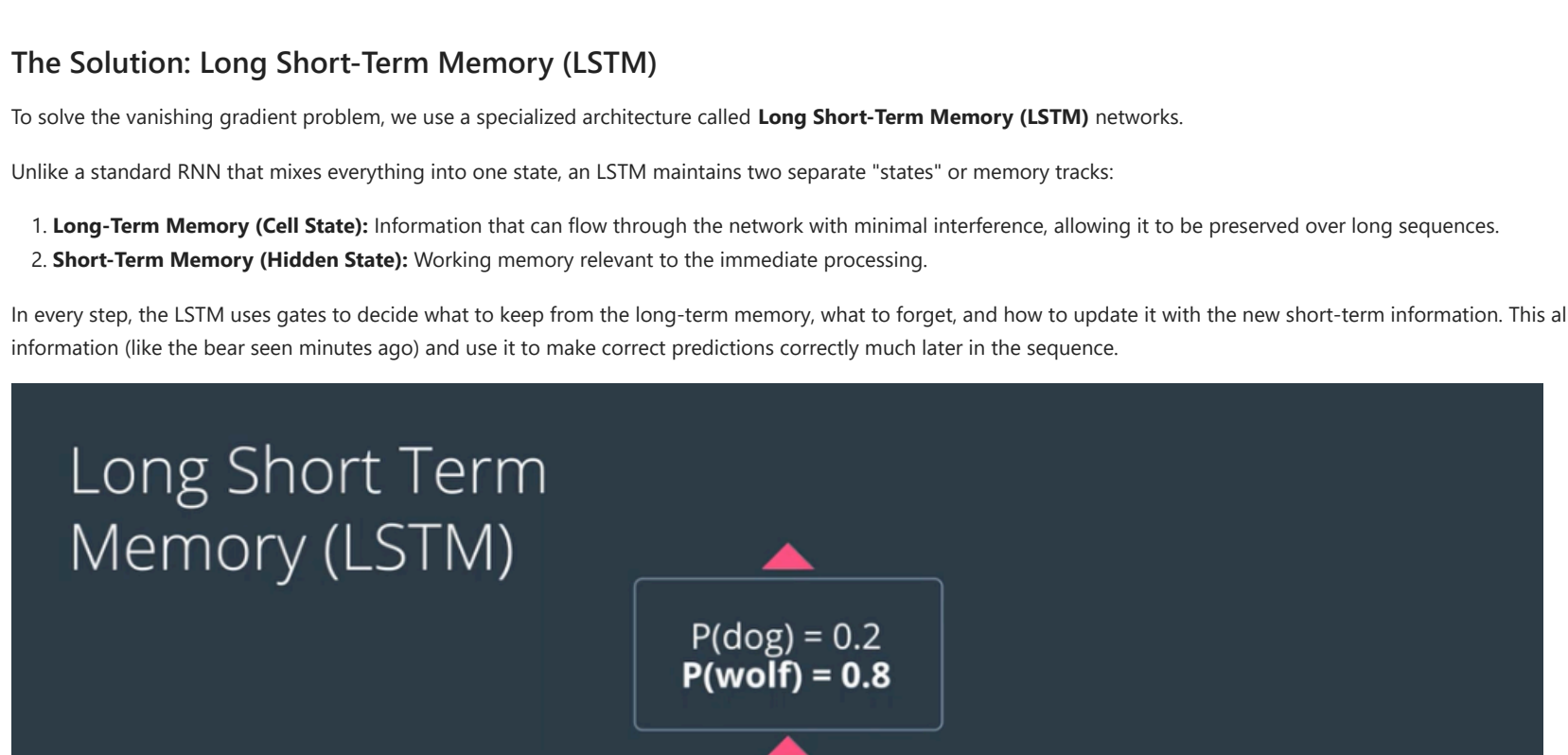
The Solution: Long Short-Term Memory (LSTM)

To solve the vanishing gradient problem, we use a specialized architecture called **Long Short-Term Memory (LSTM)** networks.

Unlike a standard RNN that mixes everything into one state, an LSTM maintains two separate "states" or memory tracks:

1. **Long-Term Memory (Cell State):** Information that can flow through the network with minimal interference, allowing it to be preserved over long sequences.
2. **Short-Term Memory (Hidden State):** Working memory relevant to the immediate processing.

In every step, the LSTM uses gates to decide what to keep from the long-term memory, what to forget, and how to update it with the new short-term information. This allows the network to "protect" vital old information (like the bear seen minutes ago) and use it to make correct predictions correctly much later in the sequence.



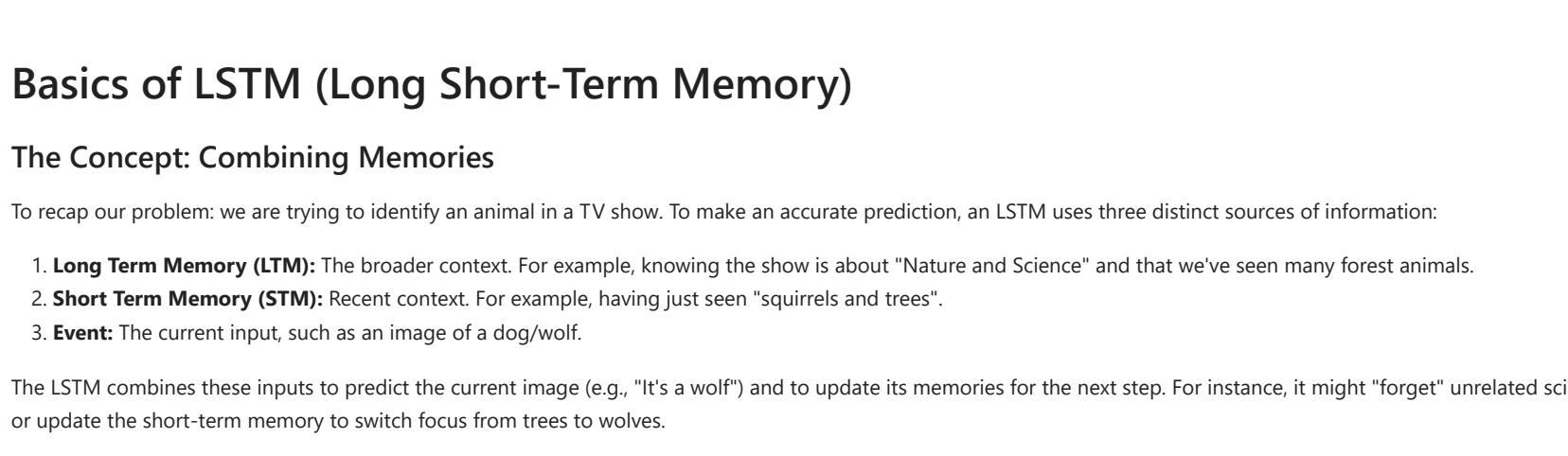
Basics of LSTM (Long Short-Term Memory)

The Concept: Combining Memories

To recap our problem: we are trying to identify an animal in a TV show. To make an accurate prediction, an LSTM uses three distinct sources of information:

1. **Long Term Memory (LTM):** The broader context. For example, knowing the show is about "Nature and Science" and that we've seen many forest animals.
2. **Short Term Memory (STM):** Recent context. For example, having just seen "squirrels and trees".
3. **Event:** The current input, such as an image of a dog/wolf.

The LSTM combines these inputs to predict the current image (e.g., "It's a wolf") and to update its memories for the next step. For instance, it might "forget" unrelated science topics but "remember" the forest context, or update the short-term memory to switch focus from trees to wolves.



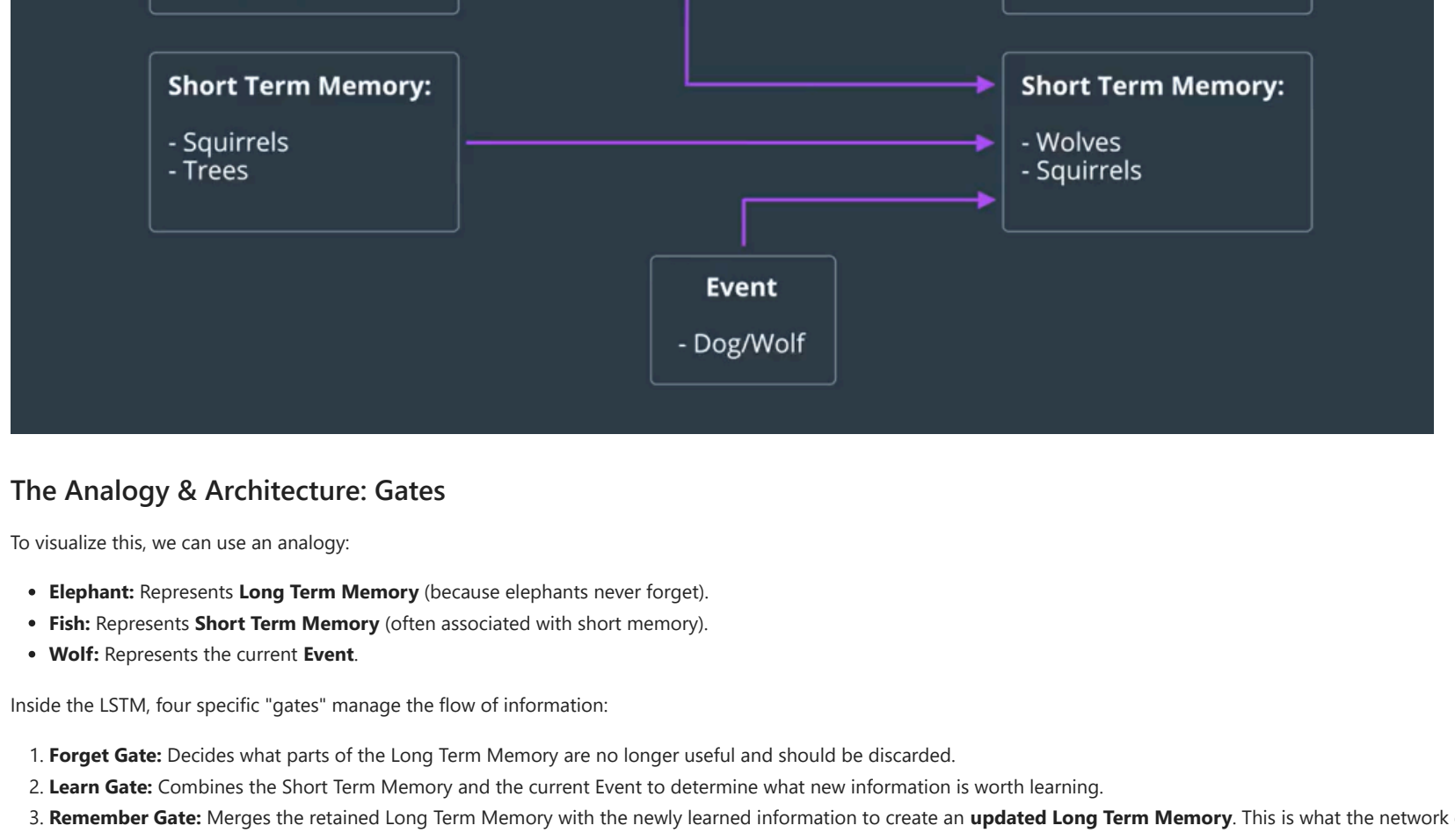
The Analogy & Architecture: Gates

To visualize this, we can use an analogy:

- **Elephant:** Represents **Long Term Memory** (because elephants never forget).
- **Fish:** Represents **Short Term Memory** (often associated with short memory).
- **Wolf:** Represents the current **Event**.

Inside the LSTM, four specific "gates" manage the flow of information:

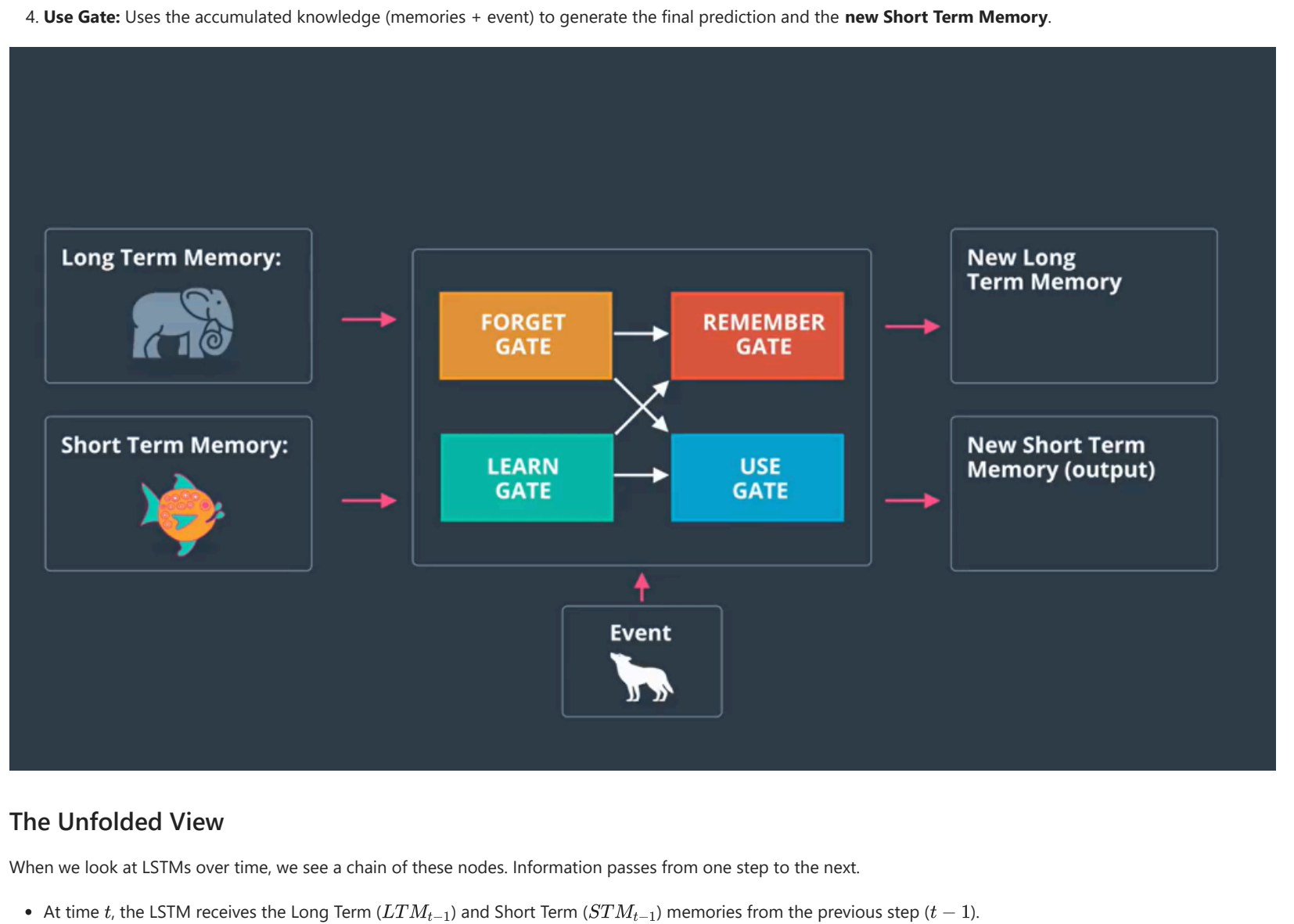
1. **Forget Gate:** Decides what parts of the Long Term Memory are no longer useful and should be discarded.
2. **Learn Gate:** Combines the Short Term Memory and the current Event to determine what new information is worth learning.
3. **Remember Gate:** Merges the retained Long Term Memory with the newly learned information to create an **updated Long Term Memory**. This is what the network carries forward to the future.
4. **Use Gate:** Uses the accumulated knowledge (memories + event) to generate the final prediction and the **new Short Term Memory**.



The Unfolded View

When we look at LSTMs over time, we see a chain of these nodes. Information passes from one step to the next.

- At time t, the LSTM receives the Long Term (LTM_{t-1}) and Short Term (STM_{t-1}) memories from the previous step ($t-1$).
- It processes them along with the current event ($Event_t$).
- It produces an output and passes updated memories (LTM_t, STM_t) to the next step ($t+1$).



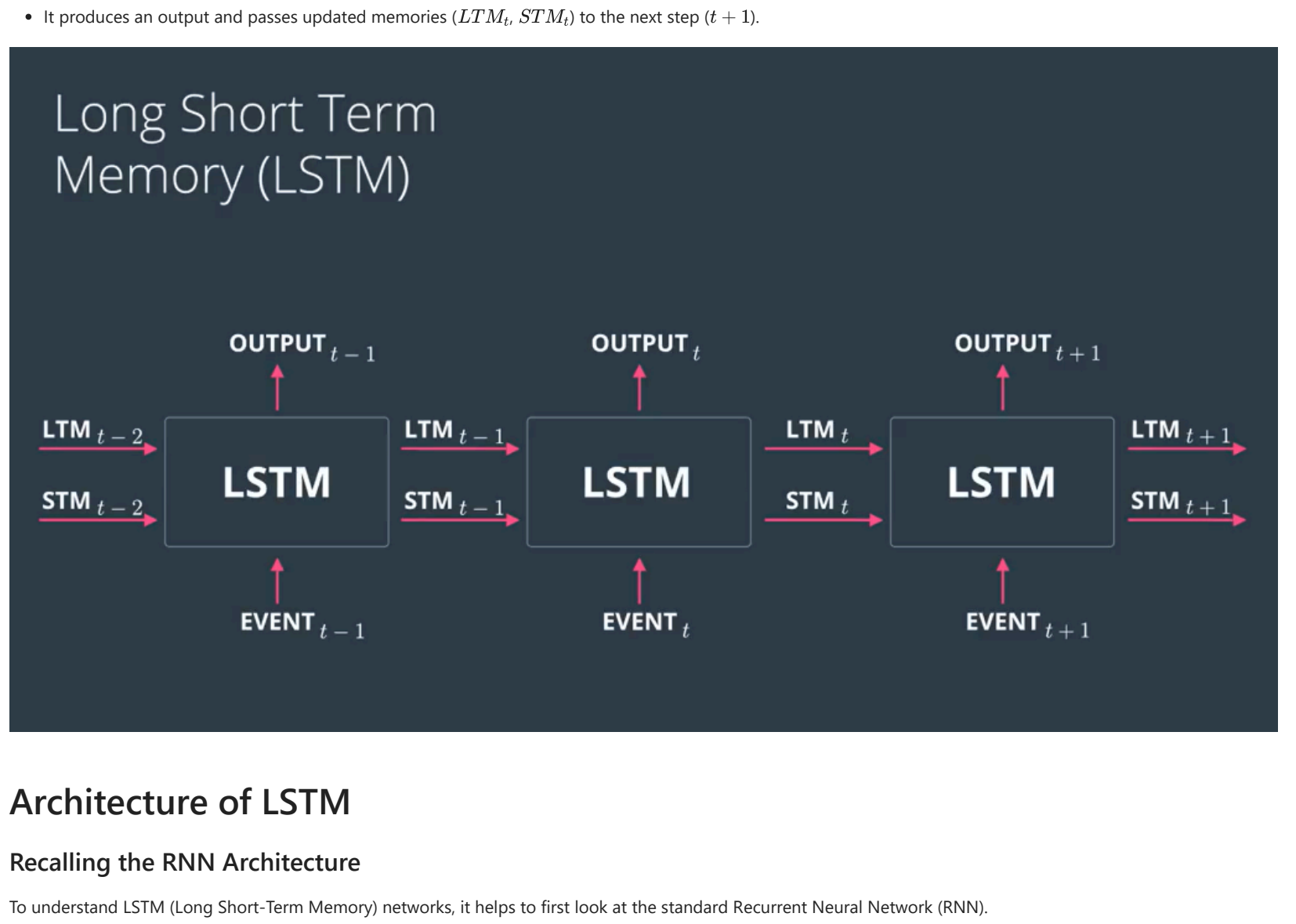
Architecture of LSTM

Recalling the RNN Architecture

To understand LSTM (Long Short-Term Memory) networks, it helps to first look at the standard Recurrent Neural Network (RNN).

In a standard RNN:

1. We have an **Event (E_t)** (new input) and **Memory (M_{t-1})** (from the previous step).
2. These vectors are combined, multiplied by a weight matrix W, and a bias b is added.
3. The result is passed through a simple activation function, typically **tanh** or **sigmoid**.
4. The output is the new **Memory (M_t)**, which serves as both the prediction for the current step and the context passed to the next step.

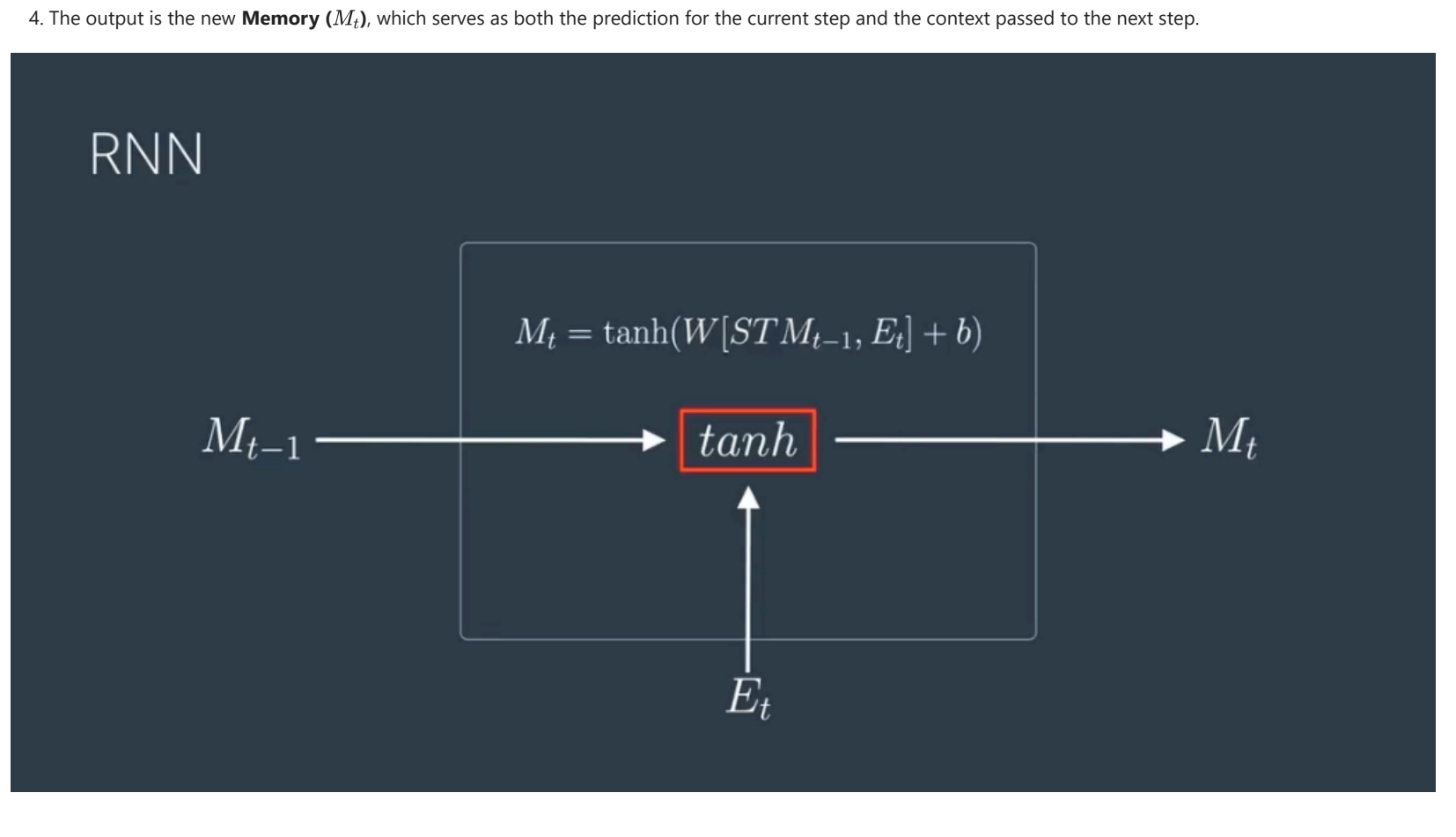


The LSTM Architecture

The LSTM follows a very similar logical flow but is more complex internally to handle long-term dependencies effectively.

1. **Inputs & Outputs:** Unlike the RNN which has one memory state, the LSTM has two:
   - **Long-Term Memory** (LTM_{t-1})
   - **Short-Term Memory** (STM_{t-1})
   - It also takes the current **Event** (E_t).
2. **Internal Gates:** Instead of a single activation layer, the LSTM contains four interacting layers (gates) that determine:
   - What to forget from long-term memory.
   - What to learn from the new event.
   - How to update the long-term memory.
   - What to output as the new short-term memory (prediction).
3. **Output:** It produces updated Long-Term Memory (LTM_t) and Short-Term Memory (STM_t). As with RNNs, the short-term memory often serves as the prediction output.

While the diagram looks complicated, it is essentially a more sophisticated version of the RNN process, designed to decide intelligently what information to keep or discard over long sequences.



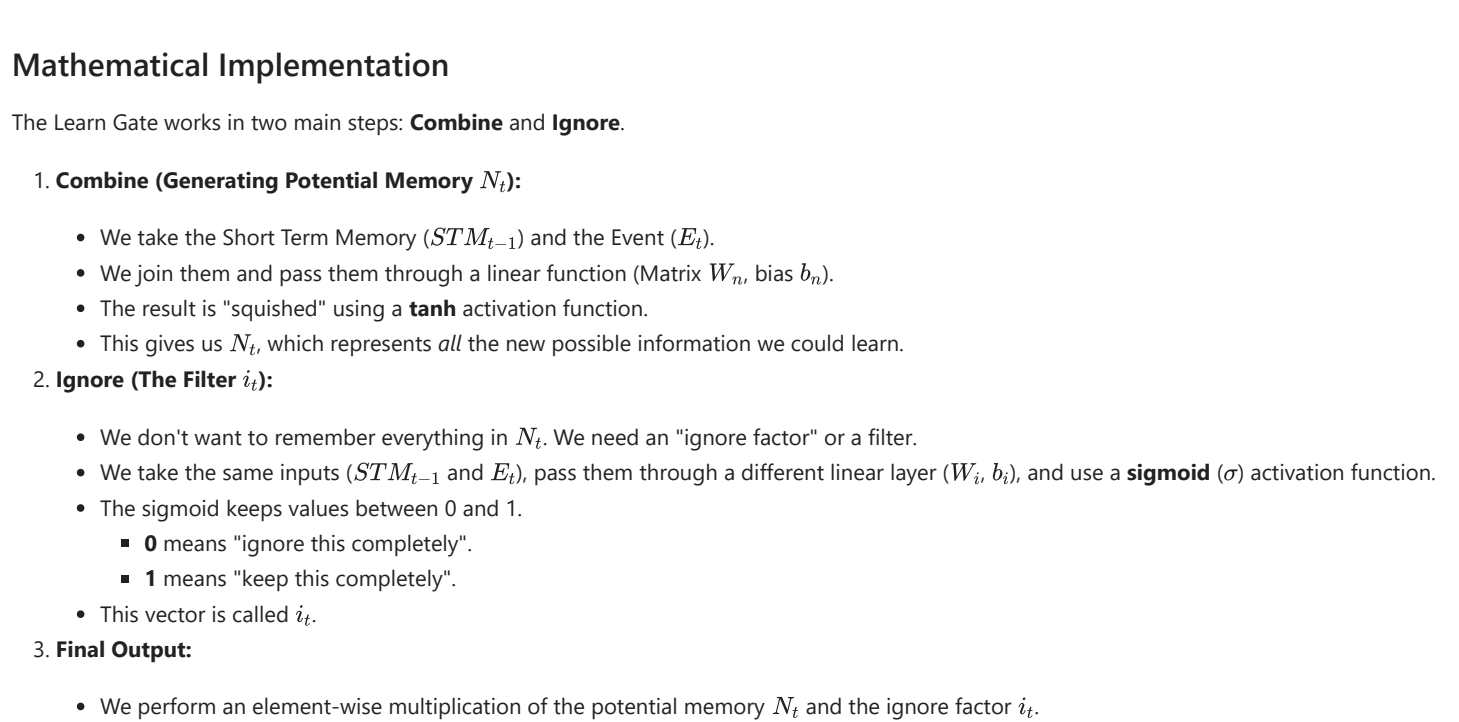
The Learn Gate

The Goal: Selectively Learning New Information

Continuing with our example:

- **Long Term Memory:** The show is about Nature/Science.
- **Short Term Memory (STM_{t-1}):** We recently saw a squirrel and a tree.
- **Event (E_t):** We see an image that looks like a dog/wolf.

The **Learn Gate** is responsible for deciding what new information from the immediate context (Short Term Memory + Current Event) should be stored. Ideally, we want to remember the "wolf" aspect (since it fits the nature context) but perhaps ignore the "tree" if it's no longer relevant.


Learn Gate
combine
STM_{t-1}
$tanh$
$N_t = \tanh(W_n[STM_{t-1}, E_t] + b_n)$
$N_t \cdot i_t$
ignore
E_t
σ
$i_t = \sigma(W_i[STM_{t-1}, E_t] + b_i)$

Mathematical Implementation

The Learn Gate works in two main steps: **Combine** and **Ignore**.

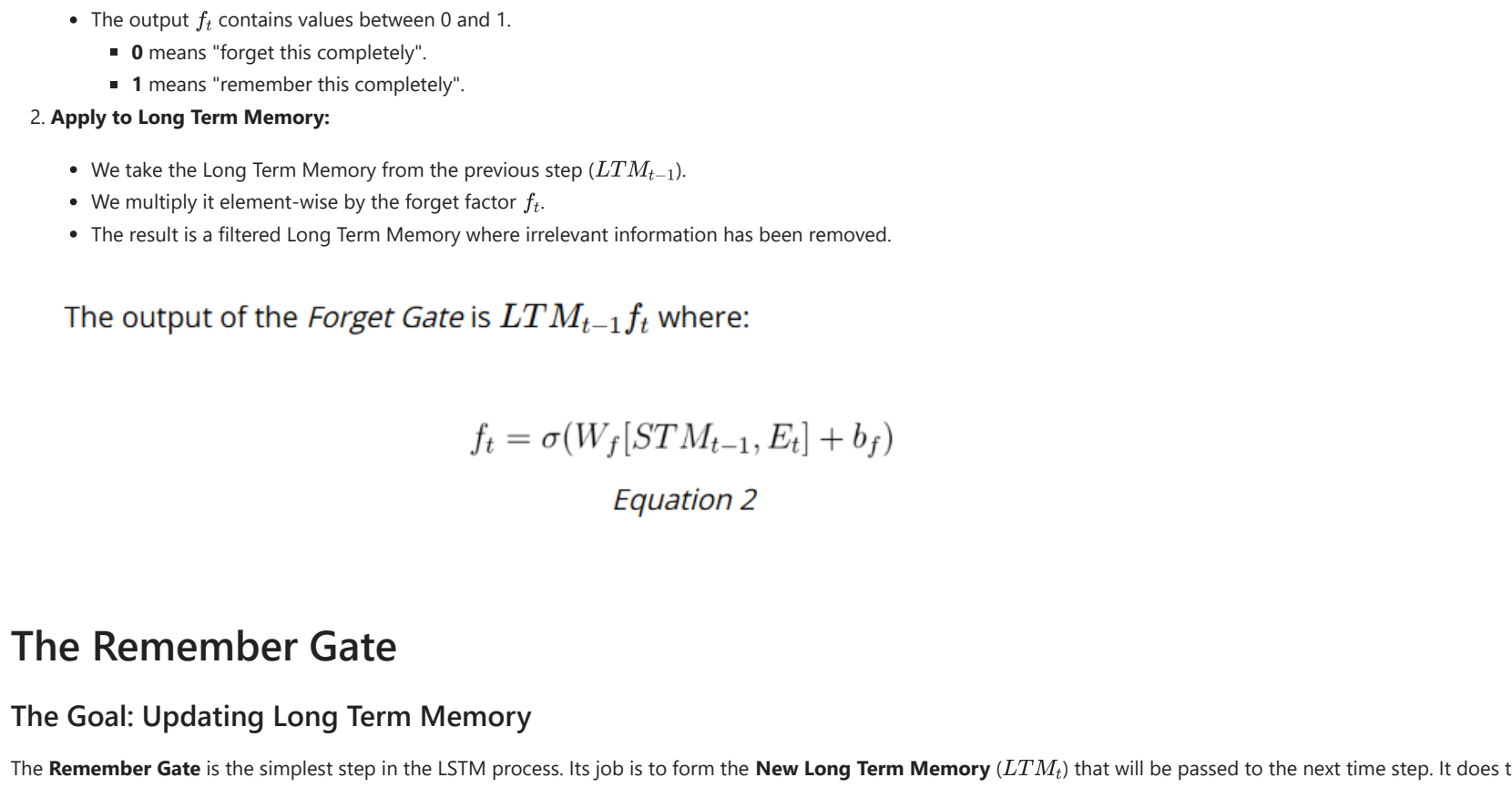
1. **Combine (Generating Potential Memory N_t):**
   - We take the Short Term Memory (STM_{t-1}) and the Event (E_t).
   - We join them and pass them through a linear function (Matrix W_n, bias b_n).
   - The result is "squished" using a **tanh** activation function.
   - This gives us N_t, which represents *all* the new possible information we could learn.
2. **Ignore (The Filter i_t):**
   - We don't want to remember everything in N_t. We need an "ignore factor" or a filter.
   - We take the same inputs (STM_{t-1} and E_t), pass them through a different linear layer (W_i, b_i), and use a **sigmoid** (σ) activation function.
   - The sigmoid keeps values between 0 and 1.
     - **0** means "ignore this completely".
     - **1** means "keep this completely".
   - This vector is called i_t.
3. **Final Output:**
   - We perform an element-wise multiplication of the potential memory N_t and the ignore factor i_t.
   - The result ($N_t \times i_t$) is the *actual* new information we learn and will use to update our memory later.

> The output of the *Learn Gate* is $N_t i_t$ where:
>
> $$N_t = \tanh(W_n[STM_{t-1}, E_t] + b_n)$$
>
> $$i_t = \sigma(W_i[STM_{t-1}, E_t] + b_i)$$
>
> *Equation 1*

The Forget Gate

The Goal: Removing Irrelevant History

The **Forget Gate** is responsible for deciding what parts of the **Long Term Memory** to discard. To continue with our example: if our long term memory stores "Science" and "Nature", but the current context suggests we should focus only on nature, the Forget Gate will help us "forget" the science part.



Mathematical Implementation

The process is straightforward:

1. **Calculate the Forget Factor (f_t):**
   - We use the **Short Term Memory** (STM_{t-1}) and the current **Event** (E_t) as inputs.
   - These are passed through a small one-layer neural network (linear function + **sigmoid** activation).
   - The output f_t contains values between 0 and 1.
     - **0** means "forget this completely".
     - **1** means "remember this completely".
2. **Apply to Long Term Memory:**
   - We take the Long Term Memory from the previous step (LTM_{t-1}).
   - We multiply it element-wise by the forget factor f_t.
   - The result is a filtered Long Term Memory where irrelevant information has been removed.

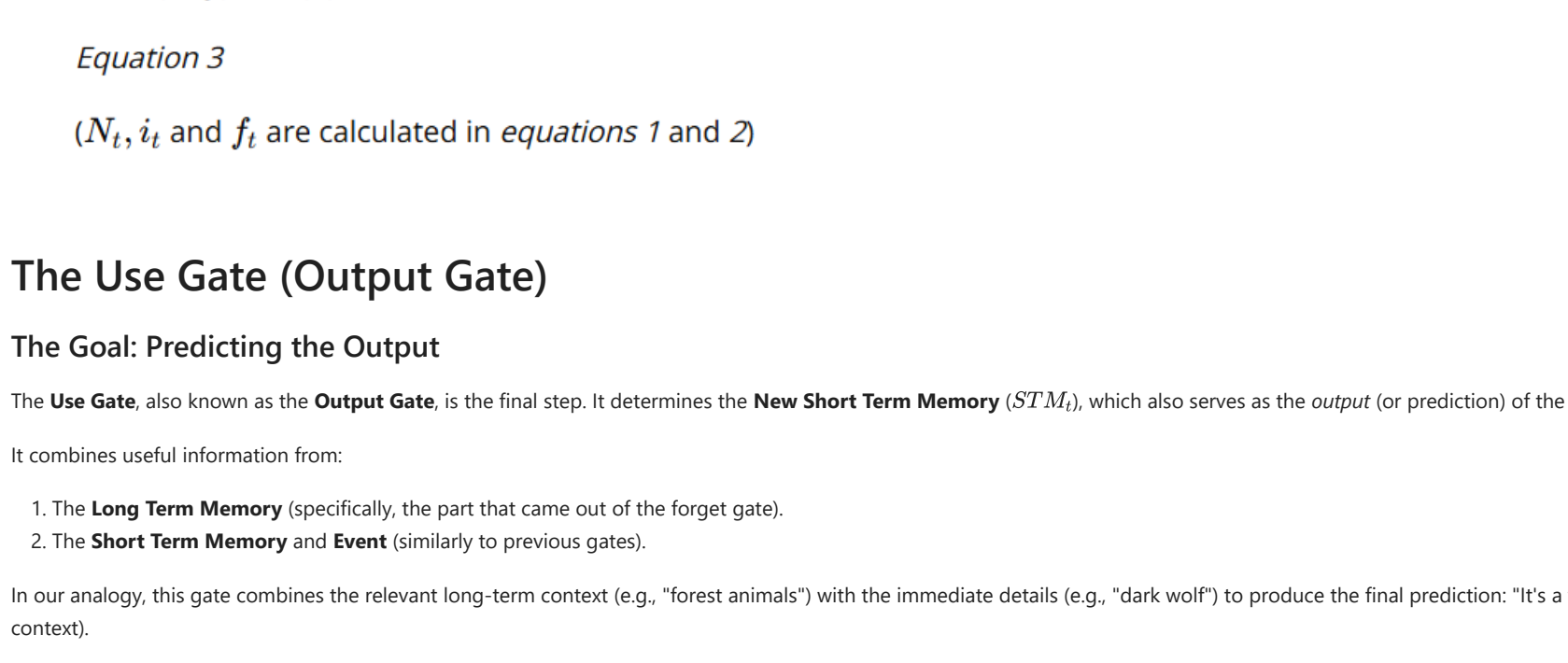
> The output of the *Forget Gate* is $LTM_{t-1} f_t$ where:
>
> $$f_t = \sigma(W_f[STM_{t-1}, E_t] + b_f)$$
>
> *Equation 2*

The Remember Gate

The Goal: Updating Long Term Memory

The **Remember Gate** is the simplest step in the LSTM process. Its job is to form the **New Long Term Memory** (LTM_t) that will be passed to the next time step. It does this by combining the results of the two previous gates:

1. The **Forget Gate** (which outputs the "remembered" part of the old long term memory).
2. The **Learn Gate** (which outputs the "new" information we decided to learn).



Mathematical Implementation

Mathematically, this is just an addition operation:

- We take the output from the Forget Gate ($LTM_{t-1} \cdot f_t$).
- We take the output from the Learn Gate ($N_t \cdot i_t$).
- We add them together to get the updated Long Term Memory LTM_t.

$$LTM_t = (LTM_{t-1} \cdot f_t) + (N_t \cdot i_t)$$

This simple addition is crucial because it allows the Long Term Memory to flow through the network with minimal transformation, helping to solve the vanishing gradient problem.

> The output of the *Remember Gate* is:
>
> $LTM_{t-1} f_t + N_t i_t$
>
> *Equation 3*
>
> (N_t, i_t and f_t are calculated in *equations 1* and *2*)

The Use Gate (Output Gate)

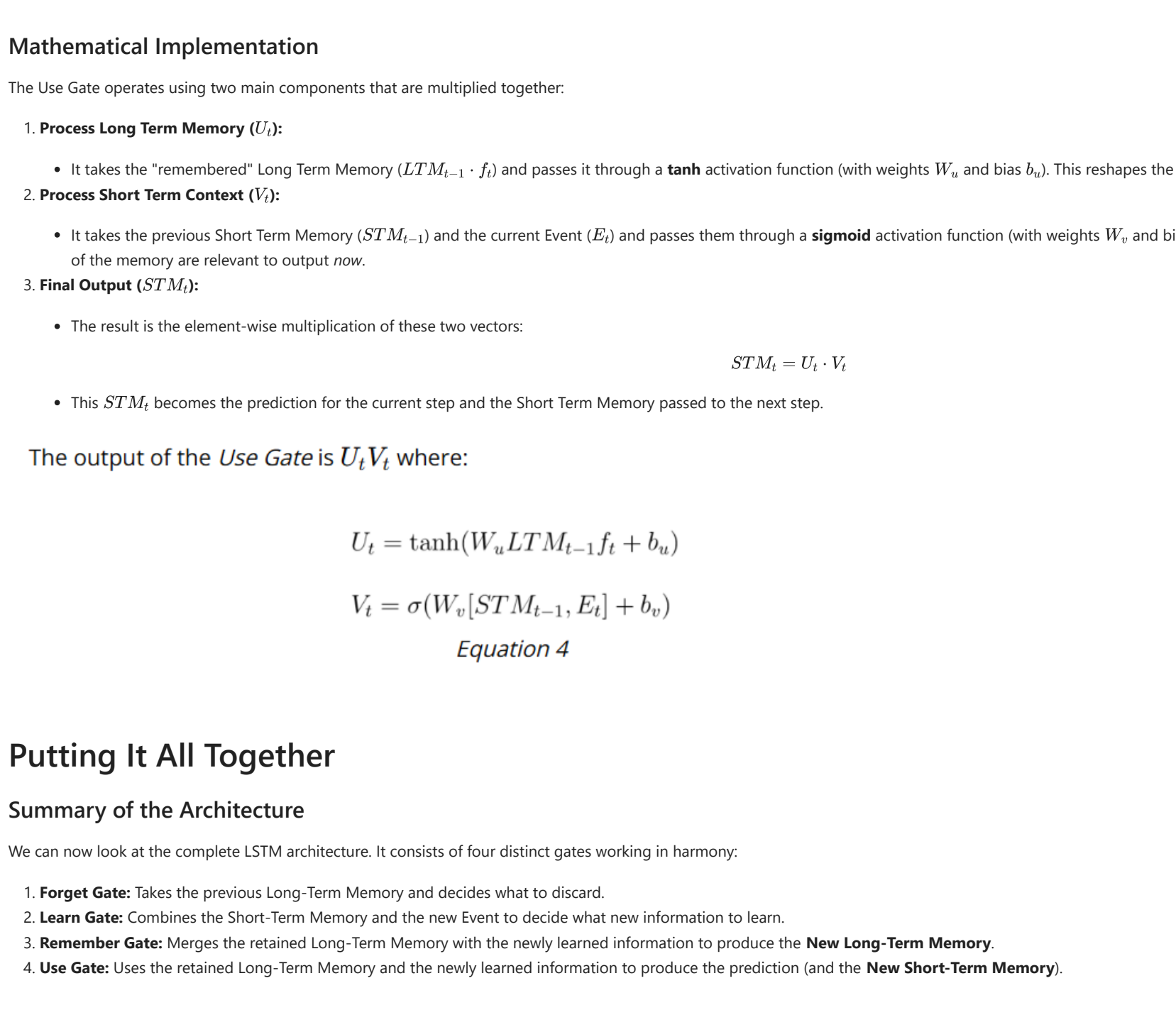
The Goal: Predicting the Output

The **Use Gate**, also known as the **Output Gate**, is the final step. It determines the **New Short Term Memory** (STM_t), which also serves as the *output* (or prediction) of the network for the current time step.

It combines useful information from:

1. The **Long Term Memory** (specifically, the part that came out of the forget gate).
2. The **Short Term Memory** and **Event** (similarly to previous gates).

In our analogy, this gate combines the relevant long-term context (e.g., "forest animals") with the immediate details (e.g., "dark wolf") to produce the final prediction: "It's a wolf" (but acknowledging the broader context).



Mathematical Implementation

The Use Gate operates using two main components that are multiplied together:

1. **Process Long Term Memory (U_t):**
   - It takes the "remembered" Long Term Memory ($LTM_{t-1} \cdot f_t$) and passes it through a **tanh** activation function (with weights W_u and bias b_u). This reshapes the values to be between -1 and 1.
2. **Process Short Term Context (V_t):**
   - It takes the previous Short Term Memory (STM_{t-1}) and the current Event (E_t) and passes them through a **sigmoid** activation function (with weights W_v and bias b_v). This acts as a filter, deciding which parts of the memory are relevant to output *now*.
3. **Final Output (STM_t):**
   - The result is the element-wise multiplication of these two vectors:

$$STM_t = U_t \cdot V_t$$

   - This STM_t becomes the prediction for the current step and the Short Term Memory passed to the next step.

> The output of the *Use Gate* is $U_t V_t$ where:
>
> $$U_t = \tanh(W_u LTM_{t-1} f_t + b_u)$$
>
> $$V_t = \sigma(W_v[STM_{t-1}, E_t] + b_v)$$
>
> *Equation 4*

Putting It All Together

Summary of the Architecture

We can now look at the complete LSTM architecture. It consists of four distinct gates working in harmony:

1. **Forget Gate:** Takes the previous Long-Term Memory and decides what to discard.
2. **Learn Gate:** Combines the Short-Term Memory and the new Event to decide what new information to learn.
3. **Remember Gate:** Merges the retained Long-Term Memory with the newly learned information to produce the **New Long-Term Memory**.
4. **Use Gate:** Uses the retained Long-Term Memory and the newly learned information to produce the prediction (and the **New Short-Term Memory**).

A Note on Arbitrariness

When looking at the full diagram, you might wonder why it is built exactly this way. Why use `tanh` here and `sigmoid` there? Why do we add in one place and multiply in another?

It admittedly looks arbitrary. You might be able to think of simpler or different architectures that seem just as logical. And you would be right—this is just one specific construction.

The primary reason the LSTM architecture is designed this way is simply because it works.

This specific configuration has proven to be extremely effective in practice. However, it is not the only way to do it. There are other variations (like GRUs) that are simpler, and the field is still evolving. Researchers are constantly experimenting with new architectures, so there is always room for innovation if you can find a structure that performs better!

Other Architectures

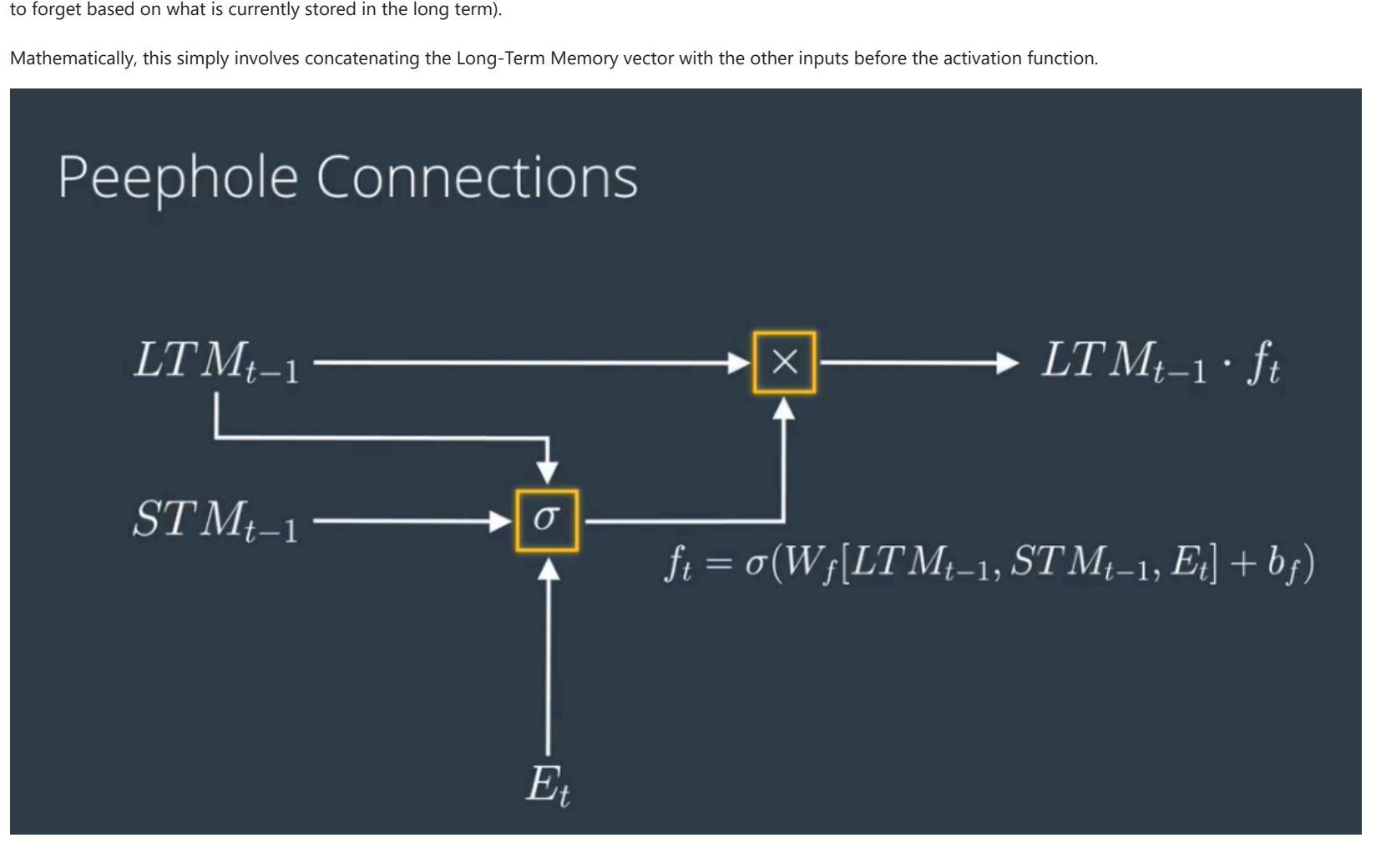
While the LSTM is a powerful architecture, it is not the only one. There are many variations, and two notable ones are the **Gated Recurrent Unit (GRU)** and the **Peephole Connection**.

Gated Recurrent Unit (GRU)

The GRU is a simpler architecture that also works very well in practice. It simplifies the LSTM by:

- Combining the **Forget Gate** and **Learn Gate** into a single **Update Gate**.
- Running the result through a **Combine Gate**.
- Merging the Short-Term and Long-Term memories into a single **Working Memory**.

This makes the network computationally lighter while often maintaining comparable performance.

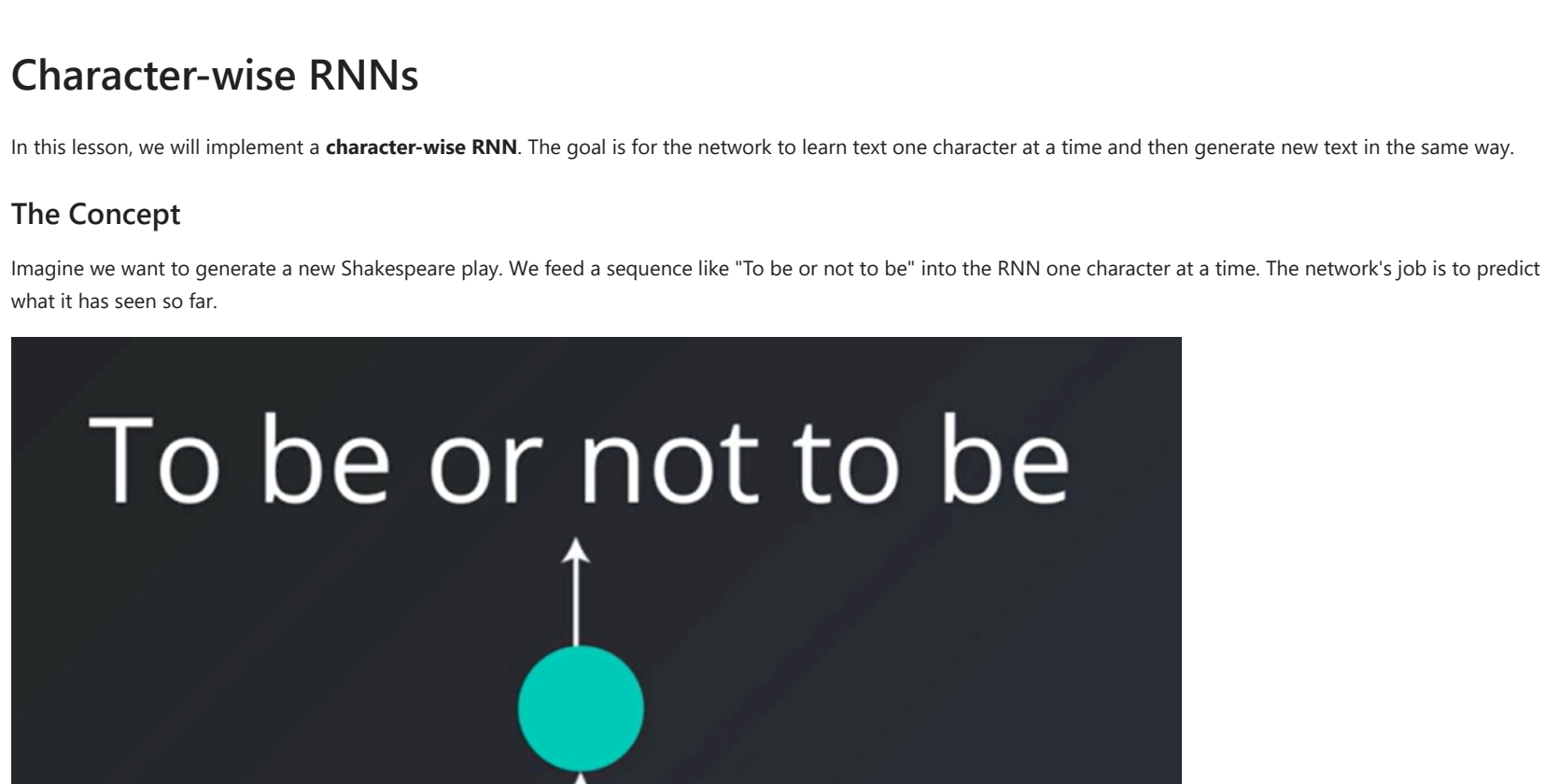


Peephole Connections

In standard LSTMs, the gates (like the Forget Gate) decide what to do based only on the Short-Term Memory and the current Event. They cannot "see" the Long-Term Memory during this decision process.

Peephole Connections fix this by allowing the Long-Term Memory to also be an input to the gate's neural network. This gives the Long-Term Memory a "say" in the decision-making process (e.g., helping decide what to forget based on what is currently stored in the long term).

Mathematically, this simply involves concatenating the Long-Term Memory vector with the other inputs before the activation function.



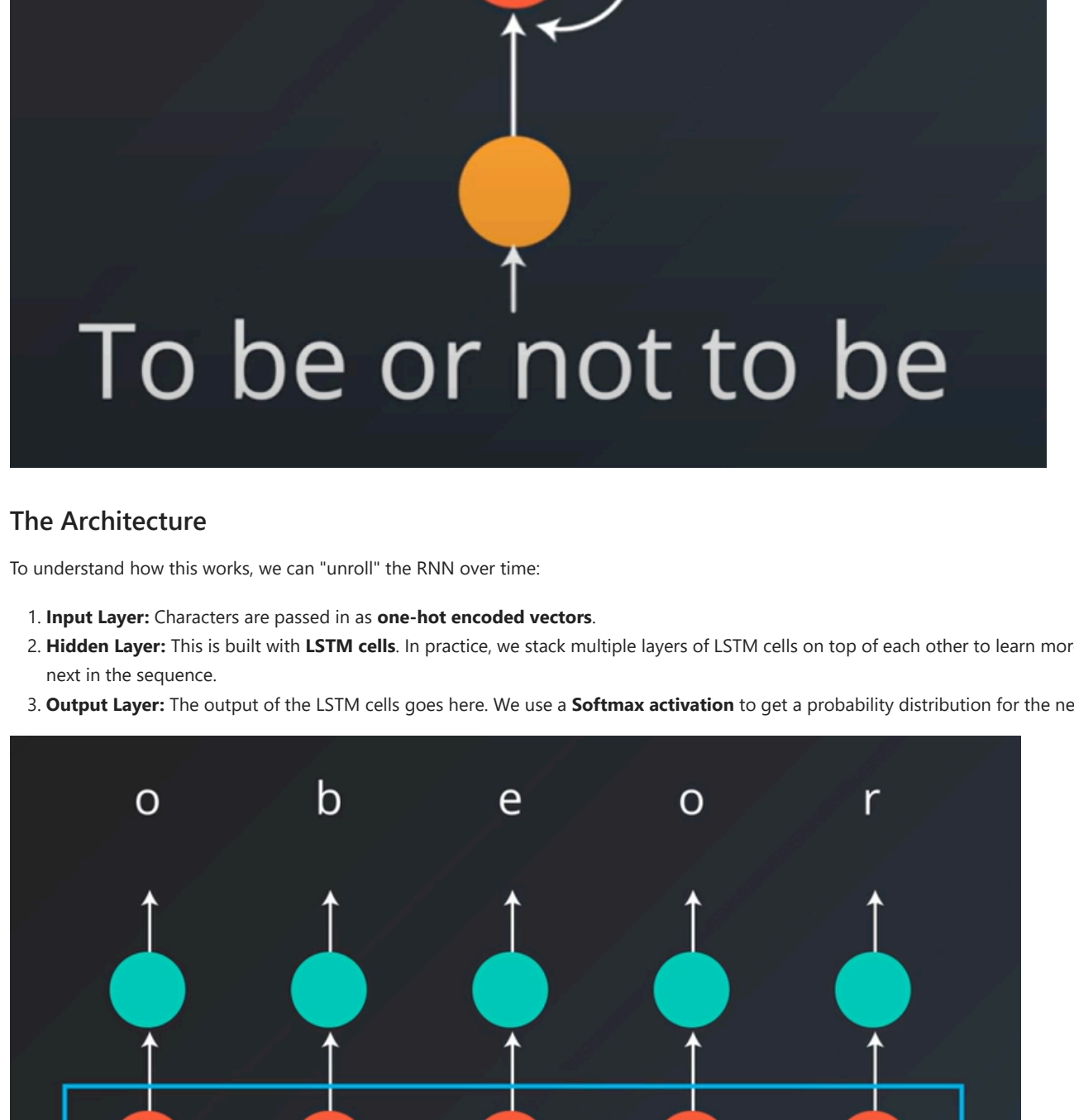
For further reading on these variations, you can refer to: Learning to Forget: Continual Prediction with LSTM

Character-wise RNNs

In this lesson, we will implement a **character-wise RNN**. The goal is for the network to learn text one character at a time and then generate new text in the same way.

The Concept

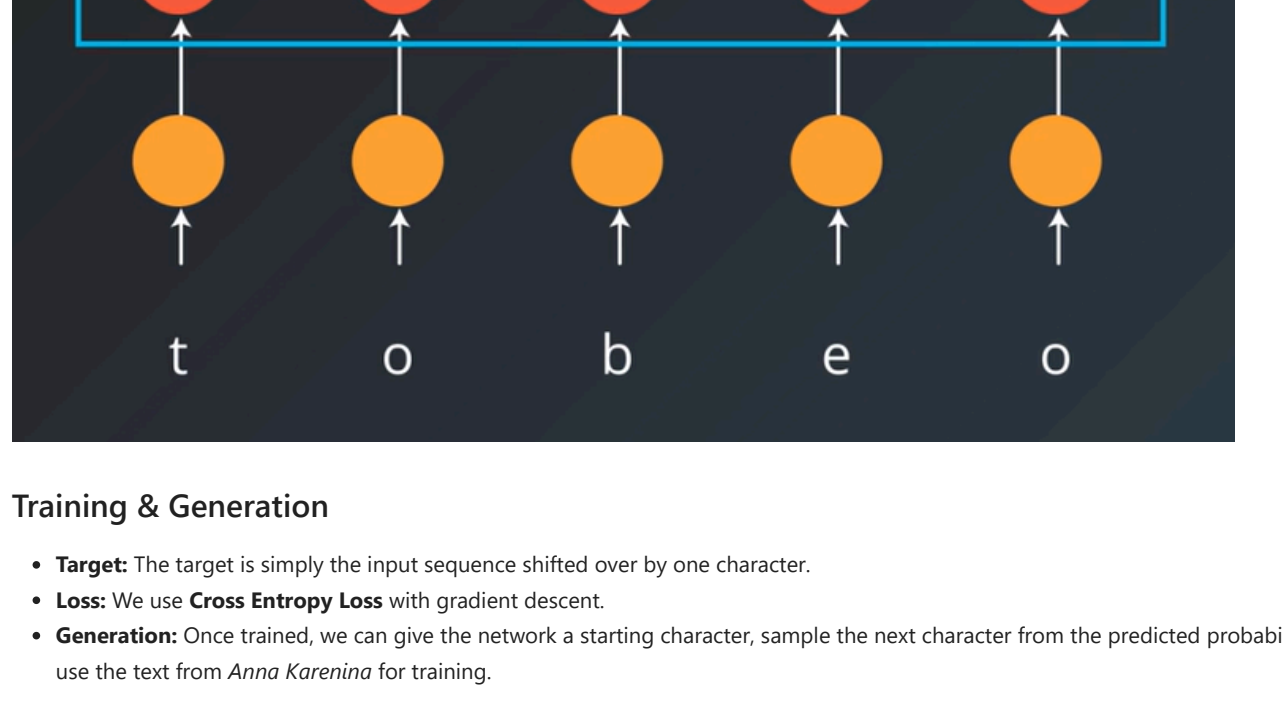
Imagine we want to generate a new Shakespeare play. We feed a sequence like "To be or not to be" into the RNN one character at a time. The network's job is to predict the *next* character in the sequence based on what it has seen so far.



The Architecture

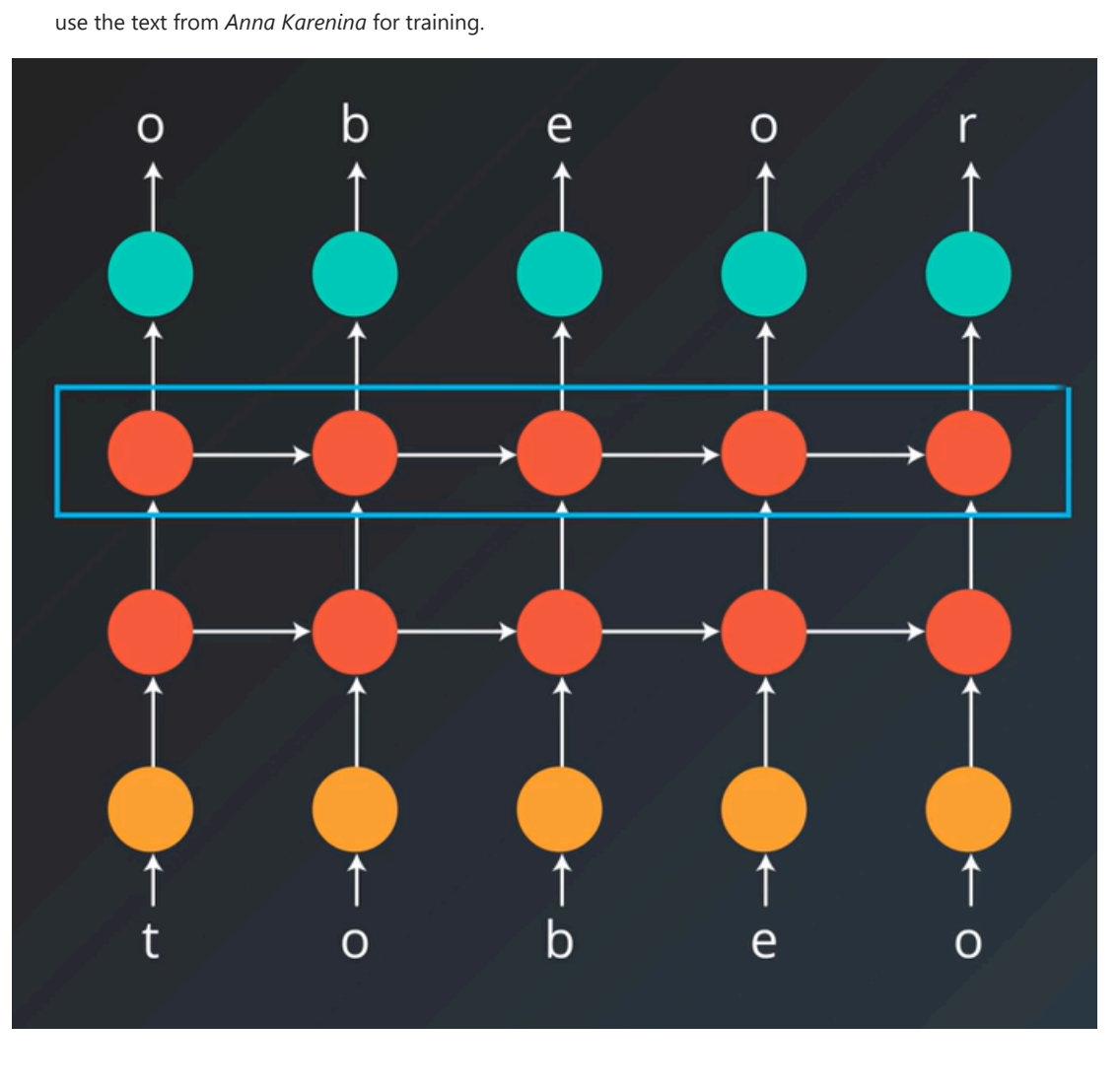
To understand how this works, we can "unroll" the RNN over time:

1. **Input Layer:** Characters are passed in as **one-hot encoded vectors**.
2. **Hidden Layer:** This is built with **LSTM cells**. In practice, we stack multiple layers of LSTM cells on top of each other to learn more complex patterns. The hidden state and cell state are passed from one cell to the next in the sequence.
3. **Output Layer:** The output of the LSTM cells goes here. We use a **Softmax activation** to get a probability distribution for the next likely character.



Training & Generation

- **Target:** The target is simply the input sequence shifted over by one character.
- **Loss:** We use **Cross Entropy Loss** with gradient descent.
- **Generation:** Once trained, we can give the network a starting character, sample the next character from the predicted probability distribution, feed that back in, and repeat to build completely new text. We will use the text from *Anna Karenina* for training.



Sequence Batching

Why Batching Matters

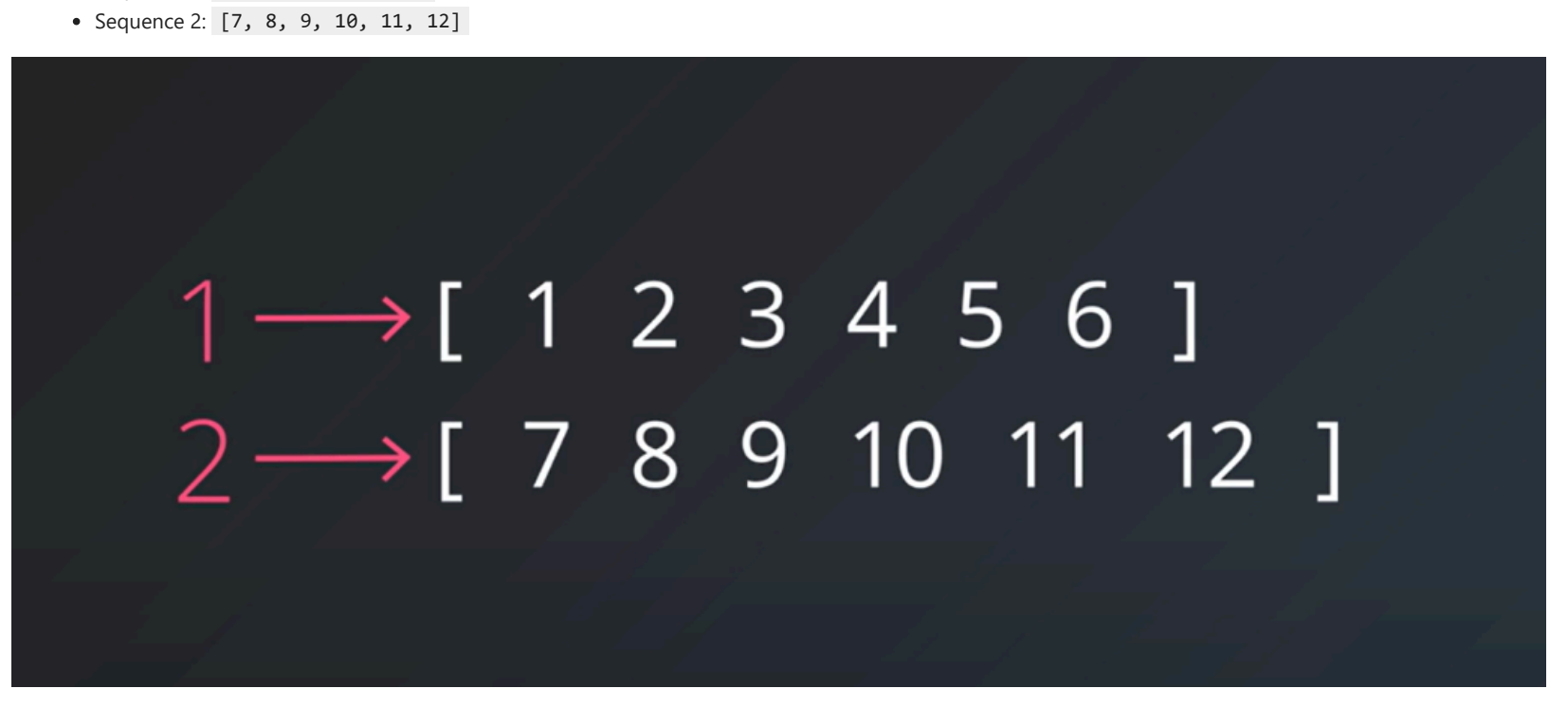
Implementing batching correctly is often more of a programming challenge than a deep learning one, but it is crucial for efficiency. RNNs train on sequences (text, audio, etc.). By processing multiple sequences in parallel, we can leverage matrix operations to speed up training.

How it Works

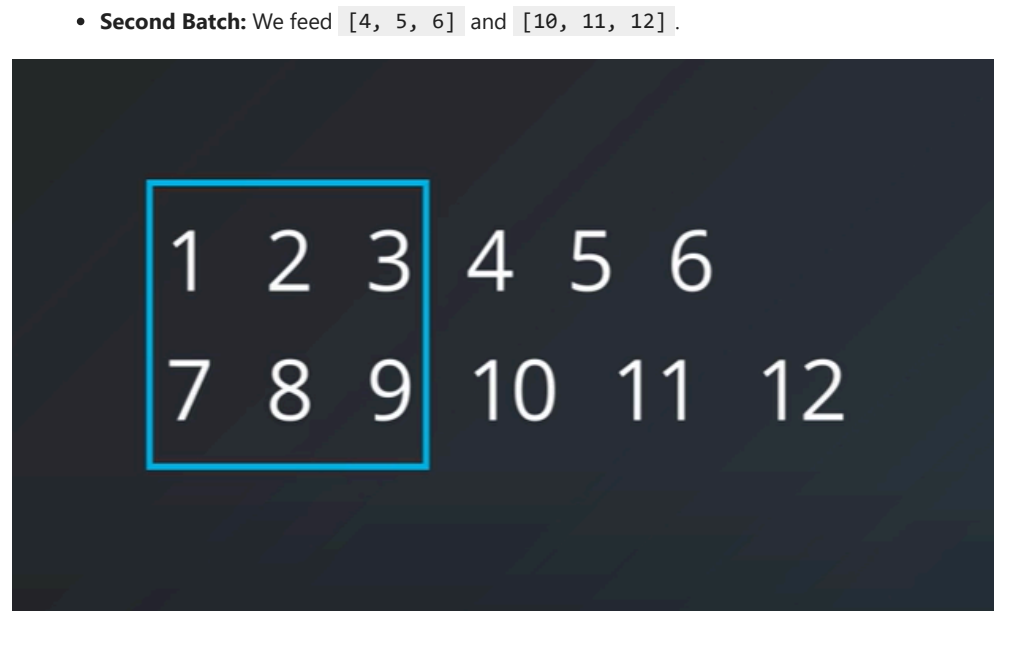
Instead of feeding one extremely long sequence into the network, we can split the data into multiple shorter sequences.

Example: Imagine we have a sequence of numbers from 1 to 12: `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]`.

1. **Batch Size:** We first decide how many parallel sequences we want. If we choose a **batch size of 2**, we split our data into two rows:
   - Sequence 1: `[1, 2, 3, 4, 5, 6]`
   - Sequence 2: `[7, 8, 9, 10, 11, 12]`



2. **Sequence Length:** Next, we decide how many steps the network sees at once. If we choose a **sequence length of 3**:
   - **First Batch:** We feed `[1, 2, 3]` and `[7, 8, 9]` into the network simultaneously.
   - **Second Batch:** We feed `[4, 5, 6]` and `[10, 11, 12]`.



State Preservation

Crucially, we retain the hidden state from the end of one batch and use it as the starting state for the next batch. This ensures that even though we are feeding data in chunks, the network still learns the continuity of the sequence (e.g., knowing that `4` follows `3`).